$\mathbb{AIM}$ Semantics for Causal Procedures

Julia Turcotti, v0.0.124

April 4, 2026

Contents

1 Preliminaries **1**
1.1 Statistical modelling . . . 2
1.1.1 Structural Causal Models . . . 2
1.2 Causal modelling . . . 3

2 $\mathbb{AIM}$ programs **3**
2.1 Grammar . . . 4
2.2 Operational Semantics . . . 4
2.2.1 $\mathbb{A}$ssign Rule . . . 5
2.2.2 $\mathbb{I}$nterve rule . . . 5
2.2.3 $\mathbb{M}$easure rule . . . 6
2.3 Functional Denotation . . . 6
2.4 Probabilistic Denotation . . . 6
2.4.1 Direct Probabilistic Denotations . . . 7
2.5 Algebraic Semantics . . . 8
2.5.1 Probability Monads . . . 8
2.5.2 ITrees . . . 9
2.5.3 Itree Equivalence . . . 9
2.5.4 Applications of ITree Equivalence to $\mathbb{AIM}$ Programs . . . 10

3 Binary Hypothesis Testing **10**
3.1 Liklihood Ratio Tests . . . 11
3.2 Neyman-Pearson Frontiers . . . 11

1 Preliminaries

This document outlines a semantics for *experimental procedures* operating against a statistical and causal model of variation amongst a population. The following subsections provide a mathematical overview of the modelling domain. Subsequent sections present a small grammar for experimental procedures ("$\mathbb{AIM}$ programs"), and provide semantics that demonstrate its expressive power for evaluating, verifying, and even synthesizing statistical estimators from their opeation.

1.1 Statistical modelling

Statistics concerns itself with answering questions about the observable characteristaics of units within a population. If we concern ourself with a finite number N (the *dimension*) of these characteristics, modelled as the *random variable*

$$X = \{X_i \in \mathcal{X}\}_{i<N}$$

then many interesting properties are immediately apparent given the *joint distribution* $p_X(\cdot) \in \mathcal{P}^X$ of X. However, the dimension of the *probability simplex* $\mathcal{P}^X$ in which this distribution lives is exponential in N, the dimension of the data, which makes modelling it directly intractable.

Intuitively, this intractability arises because the composition of individual models $\{p_{X_i}(\cdot) \in \mathcal{P}^{\mathcal{X}}\}_i$, whose modelling complexity is linear in N, suffices to model the behavior of any particular characteristic, but fails to capture the dependencies among them.

1.1.1 Structural Causal Models

Structural causal models address this inadequacy by separating the modelling of the dependencies among observable characteristics (which are common to all units) from the modelling of variation among units. In particular, variation across units is obtained by sampling from a *universe* $\{U_i\}_{i<N} \in \mathcal{U}$ with dimensionality N (as opposed to $2^{\Omega(N)}$). Since the U_i are not directly observable, but rather are strictly internal components to the model, they are also referred to as the *unobserved* terms.

In general, if nothing further is known about the dependencies amongst the X_i, then the task of modelling X from $\mathcal{U}$ is still $2^{\Omega(N)}$-intractable. However in practice, each X_i can usually be reduced to dependency on its own corresponding U_i, and on a small (i.e. $o(N)$) number of other X_i, yielding $2^{o(N)}$ overall modelling complexity. In particular, there usually exists a parameter set $\Theta : |\Theta| = 2^{o(N)}$, each of whose terms specifies one possible form the limited dependencies amongst the X_i can take. A given choice of $\theta \in \Theta$ fixes the observable behaviors of each universe relative to its unobserved terms, so is henceforth referred to as a *meta-universe*.

Formally, X, U, and θ are related via an *observation operator*

$$\mathcal{O}_i : \mathcal{U} \times \Theta \to \mathcal{X}$$

which computes the observable term X_i for a fixed choice of universe and meta-universe. Together, the triple

$$\Gamma_s := (\mathcal{U}, \Theta, \mathcal{O})$$

forms a fully general (statistical) model of X (with feasible dimensionality!).

1.2 Causal modelling

Separating the modelling of X into the modelling of a universe U and a meta-universe θ is a major win not just in terms of modelling efficiency, but in terms of expressive power of the model. Indeed, the standard statistical operations of conditioning and measurement (which concern observations and inference of the universe $\mathcal{U}$) can be extended with *causal* operations, such as *intervention*, that have a common effect across all universes that can be directly modelled as observations and conditioning on the meta-universal parameter θ.

For example, small local changes to the dependency structure, such as intervention, which removes all extrinisic dependencies of X_i and renders X_i (across all universes) dependent only on the nature of the intervention, are easily modelled as (computationally inexpensive e.g. $O(1)$) changes denoted by the operator

$$\mathcal{I}_i^x : \Theta \to \Theta$$

which observes that a given intervention is, denotationally, just a self-mapping within the domain of meta-universes. Equivalently, we can note that the action of an intervention is fully characterized as a mutation in the underlying choice of meta-parameter θ. Now, the quadruple

$$\Gamma := (\mathcal{U}, \Theta, \mathcal{O}, \mathcal{I})$$

forms a feasibly-dimensional fully-general model of not just the universal variation amongst our population, but the *meta-universal* variation amongst the many plausible underlying mechnanisms determining the nature of that variance. More concisely, Γ extends Γ_s to be not just a *statistical* model, but a *causal* model.

Causal modelling allows for the expression of a strictly richer experimental semantics than statistical modelling, as will be expanded upon in the sequel.

2 $\mathbb{AIM}$ programs

Rarely do queries of interest about a model of a population take the form of simple, "atomic", probabilistic queries such as $\mathbb{P}[X_i = x] \sim p_{X_i}(x)$. In general, queries of interest often depend not just on more general dependencies amongst the observable data[1], but on the semantics of *causal* operations such as intervention[2]

[1] though many interesting statistical properties do take this form, e.g. $\mathbb{P}[X_i + X_j > X_k | X_l > X_m]$

[2] consider $|\mathbb{P}[X_i | do(X_j) = x] - \mathbb{P}[X_i | X_j = x]| > \epsilon]$ - a query which can answer the question "can populations that are naturally observed to obey $X_j = x$ be distinguished from populations in which $X_j = x$ is artifically enforced by observing X_i?", for example it is known statistically that higher performing grade school students eat breakfast, but can we further conclude (causally) that providing all students with breakfast will lead to uniformly higher performance?).

2.1 Grammar

To address experimental procedures that can interact with populations in statistical, casual, and mixed modes, the following grammar is presented:

$$\begin{aligned} \text{Term } \mathcal{T} &:= \mathbb{A}_i^{\mathcal{X}} \mid \mathbb{I}_i^{\mathcal{X}} \mid \mathbb{M}_i \\ \text{Program } \mathcal{R} &:= \epsilon \mid \mathcal{T}; \mathcal{R} \\ \text{Result } \mathcal{D} &:= \epsilon \mid \mathcal{X}; \mathcal{D} \end{aligned}$$

$\mathcal{T}$ is an $\mathbb{AIM}$ term (whose semantics will shortly be explicated)[3], $\mathcal{R}$, an $\mathbb{AIM}$ program, is a sequence of zero or more terms ($\mathcal{R} = \mathcal{T}^*$), and $\mathcal{D}$, an $\mathbb{AIM}$ result, is a sequence of zero or more values from the domain $\mathcal{X}$ of our random variables X_i.

To briefly motivate this term language, we provide the following interpretations of the three operators:

$\mathbb{A}_i^x$: assign – only proceed with execution if $X_i = x$

$\mathbb{I}_i^x$: intervene – use causal intervention to enforce $X_i = x$ for the remainder of execution

$\mathbb{M}_i$: measure – include the present value of X_i in the output

2.2 Operational Semantics

An operational semantics interprets programs as compositional transforms on state. Arguably, this is the presentation of semantics that maps most directly to programmer intuition, and aligns most closely with some of our most fundamental models of execution; like Turing machines. We thus present an operational semantics below for $\mathbb{AIM}$ as way to ground the intuition for their meaning presented above.

This operational semantics forms the core of the enumerative boolean engine we implemented in the accompanying artifact `../operationalize.ml`. For more realistic domains, enumerative exploration of the state transforms defined by operational semantics is infeasible. Thus, in subsequent sections, we explore more direct (denotational/axiomatic) semantics for $\mathbb{AIM}$ programs that are less intuitive (the presentation *requires* intutition to use rather than `builds` intuition to use), but lend themselves more directly to scalable implementation.

Recall $\Gamma := (\mathcal{U}, \Theta, \mathcal{O}, \mathcal{I})$ specifies a choice of universe, meta-universe, and operations for observation and intervention. Taking $\mathcal{O}$ and $\mathcal{I}$ as fixed artifacts of the data representation, the following judgement form suffices to encode an *evaluation semantics* (i.e. operational semantics) for $\mathbb{AIM}$ programs:

[3] in its current form, $\mathbb{AIM}$ programs can be reduced to equivalent $\mathbb{IM}$ expressions with output post-processing (i.e. the $\mathbb{A}$ operator is redundant), but but the current $\mathbb{A}$ construct can be naturally extended to a non-redundant branching construct in future forms

$$\mathcal{U}, \Theta \vdash \mathcal{R} \downarrow \mathcal{D}$$

Interpreted as of "in universe $\mathcal{U}$, in meta-universe Θ, $\mathbb{AIM}$ program $\mathcal{R}$ evaluates to result $\mathcal{D}$", and is derived as follows:

2.2.1 $\mathbb{A}$ssign Rule

$$(\mathbb{A}^{+}\text{-eval}) \quad \frac{\mathcal{U}, \Theta \vdash \mathcal{R} \downarrow \mathcal{D} \quad \mathcal{O}_i(\mathcal{U}, \Theta) = x}{\mathcal{U}, \Theta \vdash \mathbb{A}_i^x; \mathcal{R} \downarrow \mathcal{D}}$$

$$(\mathbb{A}^{-}\text{-eval}) \quad \frac{\mathcal{O}_i(\mathcal{U}, \Theta) \neq x}{\mathcal{U}, \Theta \vdash \mathbb{A}_i^x; \mathcal{R} \downarrow \epsilon}$$

These two rules define the semantics of the $\mathbb{A}$ "assign" term. Intuitively, a program containing a particular $\mathbb{A}$ term at point $\mathcal{R}_0; \mathbb{A}_i^x; \mathcal{R}_1$ evaluates to the result of $\mathcal{R}_0; \mathcal{R}_1$ if $X_i = x$ at that point, and to just the result of $\mathcal{R}_0$ otherwise. In this sense it is functionally equivalent to a conditional "abort" or "assert". In the context of a broader scientific trial, it is best interpreted as "filtering" the involved participants (or other units of study).

It also serves as a primitive placeholder for more sophisticated branching constructs TBA.

2.2.2 $\mathbb{I}$nterve rule

$$(\mathbb{I}\text{-eval}) \quad \frac{\mathcal{U}, \mathcal{I}_i^x(\Theta) \vdash \mathcal{R} \downarrow \mathcal{D}}{\mathcal{U}, \Theta \vdash \mathbb{I}_i^x; \mathcal{R} \downarrow \mathcal{D}}$$

This rule defines the semantics of the $\mathbb{I}$ "intervene" term. Intuitively, a program $\mathcal{R}_0; \mathbb{I}_i^x; \mathcal{R}_1$ returns a result in which the meta-universe in which the sequel $\mathcal{R}_1$ is evaluated differs from that in which $\mathcal{R}_0$ by an application of the intervention operator $\mathcal{I}_i^x \downarrow \Theta \to \Theta$ - which "forces" X_i to take value x in the remainder of the program.

This highlights the expressive power of the separation of modelling concerns into a statistical universe $\mathcal{U}$ and a casual meta-universe Θ. It may be more intuitively obvious that an experiment can perform inference about a fixed meta-universal paramter θ by running repeated trials across differing universal draws $\mathcal{U}$ - e.g. by obtaining many distinct participants in the trials - but the semantics of the $\mathbb{I}$ term highlight that fixing $\mathcal{U}$, and varying Θ - e.g. by performing measurements before and after structural interventions in the causal mechanisms under study - is equally expressible. Further, evaluation will show that the ability to separately vary both $\mathcal{U}$ and Θ provides strict improvements in inferential power.

2.2.3 $\mathbb{M}$easure rule

$$(\mathbb{M}\text{-eval}) \quad \frac{\mathcal{U}, \Theta \vdash \mathcal{R} \downarrow \mathcal{D}}{\mathcal{U}, \Theta \vdash \mathbb{M}_i; \mathcal{R} \downarrow \mathcal{O}_i(\mathcal{U}, \Theta); \mathcal{D}}$$

Finally this rule defines of the $\mathbb{M}$ "measure" term. perhaps the most straightforward of the three $\mathbb{AIM}$ terms, $\mathcal{R}_0; \mathbb{M}_i; \mathcal{R}_1$ computes the value of X_i post-evaluation of $\mathcal{R}_0$, and appends that value to the running stack of results.

In particular, we can now note that the measurable result $\mathcal{D}$ of a program $\mathcal{R}$ that terminates naturally (i.e. can be evaluated without invoking the $\mathbb{A}^-$ "abort" rule) is a vector of values from $\mathcal{X}$ equal in length to the number of $\mathbb{M}$ terms in the program. If evaluation instead invokes the abort rule at program point $\mathcal{R} = \mathcal{R}_0; \mathbb{A}; \mathcal{R}_1$, then the result $\mathcal{D}$ is a vector of values equal in length to the number of $\mathbb{M}$ terms in $\mathcal{R}_0$ (i.e. equal to the result of $\mathcal{R}_0$).

2.3 Functional Denotation

The above semantics, inhabiting the judgement $\mathcal{U}, \Theta \vdash \mathcal{R} \downarrow \mathcal{D}$, suffices to characterize the behavior of $\mathbb{AIM}$ programs as computationally explicit *functions*:

$$\mathbb{F}(\mathcal{R}) : (\mathcal{U}, \Theta) \to \mathcal{D}$$

These functions can be evaluted "forwards" to translate known characteristics of $\mathcal{U}$ and Θ (such as knowledge of (possibly probabilistic) properties of units (universes) or of the underlying causal mechanisms (meta-universes)) into characteristics of the result $\mathcal{D}$ of running $\mathcal{R}$. For example, that knowing a certain causal link is absent suffices to guarantee the independence of the results two experimental procuedures.

However, the goals of statistical inference (of $\mathcal{U}$) or causal inference (of Θ) are more directly tied with the "backwards" semantics of these function $\mathbb{F}(\mathcal{R})$. Namely, how can we improve on partial (or nil) information about $(\mathcal{U}, \Theta)$ by observing $\mathbb{F}(\mathcal{R})(\mathcal{U}, \Theta)$? In particular, the nature of our split universal/meta-universal modelling means most learning objectives are well-described as trying to define *estimators* or *belief updates* for Θ given a dataset composed of the result of $\mathcal{R}$ for many successive draws of $\mathcal{U}$.

2.4 Probabilistic Denotation

By introducing a "prior" on $\mathcal{U}$[4] we can now characterize our task as the modelling of the following markov chain:

$$\Theta \to \mathcal{D} \to \hat{\Theta}$$

[4]this may seem to be a significant modelling obstacle, but in practice the maxim of "max entropy priors" suffices to provide a stable choice of p_U near the "center" of the simplex $\mathcal{P}^{\mathcal{U}}$ that is near-optimal for most inferential procedures

Where $\mathbb{P}[\mathcal{D} \mid \Theta]$ is given by $\Sigma_u\{p_U(u) \mid \mathbb{F}(\mathcal{R})(u, \Theta) = \mathcal{D}\}$, and appropriate definition of $\mathbb{P}[\hat{\Theta} \mid \mathcal{D}]$ (that, notably, cannot depend on $\mathcal{U}$ or Θ) characterises the concern of modelling the "backwards" direction of $\mathbb{F}(\mathcal{R})$.

In statistical terminology, $\hat{\Theta}$ is referred to as an *estimator*. Notable choices of $\hat{\Theta}$ include the maximum liklihood (ML) estimator, the minimax estimator, and, for the particualar case of answering binary queries about Θ, the *liklihood ratio tests* (expanded upon and implemented for a special case below). This setting also defines many meaingful performance criteria for estimators that aid search and verification, most notably, the *channel capacity* (Shannon capacity) of the direct channel $\Theta \to \mathcal{D}$, or even the capacity of induced channels like $\mathcal{D}_1 \to \mathcal{D}_2$ - i.e. trying to infer the result of one program by observing the result of another.

Each of estimators and quantities above is called a "non-bayesian" because it does not rely on a prior for Θ[5]. If, however, a prior on Θ *is* available, then further techniques (like Bayesian least squares, min log loss, etc) become available, and often reduce to tractable convex optimizations.

Our artifact `../operationalize.ml` relies on the specific case of implementing *likelihood ratio tests*, but the reduction of a broad class of qualitiative modelling concerns to known procedures on the conditional distributions (referred to in statistics as the "modelling family") $\mathcal{P} = \{p_{\mathcal{D};\Theta}(\cdot;\theta)\}_{\theta\in\Theta}$[6] provides a much greater set of tools of relevance to the overarching aim of experimental analysis

2.4.1 Direct Probabilistic Denotations

To obtain a characterization of $\mathcal{P}$ from the functional denotations $\mathbb{F}(\mathcal{R})$ requires a multiply exponential enumeration[7] that is only computationally tractable for "toy" domains. Since its correctness depends on nothing but acceptance of the judgement rules $\mathcal{U}, \Theta \vdash \mathcal{R} : \mathcal{D}$ and basic probabilistic identities, it is still useful as *reference implementation*, but must be shown to correspond to a more efficient implementation to be of practical use.

Without delving too far into this concern, we note that $\mathbb{AIM}$ programs can alternatively be given a direct probabilistic denotation in the "do"-augmented probability calculus of the following form:

$$\forall \mathcal{D}, \mathbb{P}[\mathcal{R} : \mathcal{D}] \sim \mathbb{P}[\langle f(X_i) | [f(X_i)]\rangle \mid [\langle f(X_i) | [f(X_i)]\rangle]]$$

Where braket notation expresses the "do" operator and several other grammatical constructs are used without explanation. Further work will thisupon expound.

[5] which, unlike $\mathcal{U}$, often does not lie within a euclidean domain like $\mathbb{R}^K$ that provdes meangingful max-entropy priors

[6] or $\{p_{\mathcal{D}|\Theta}(\cdot \mid \theta)\}_{\theta\in\Theta}$ in the bayesian case

[7] though probabilistic approximations such as Monte Carlo sampling, and indeed any of the many techniques used for the generalization fo this task to inference for modern *probabilistic programs*, are all of import

2.5 Algebraic Semantics

[Section WIP - Author's understanding of mentioned concepts still evolving]

The prior subsection investigated the denotationalization of $\mathbb{AIM}$ programs through embeddings into the space of transforms on probability distributions ($\mathcal{U} \times \Theta \to \mathcal{P}^{\mathcal{D}}$).

In the worst case scenario for mechanized analysis, these transforms are represented with an axiomitization of the reals and of real analysis (see `math-comp/analysis` library on github).

More realistically, proofs could rely on a presentation of distributions and transforms on distributions that exposes a sufficient set of "basic" facts about probability to reason, in most cases, only through those facts (see `affeldt-aist/infotheo` or `EasyCrypt/certicrypt`).

However, even a program semantics that reduces to intuitive algebraic transforms on distributions is far more general that is needed to analyze probabilistic programs as simple as $\mathbb{AIM}$ programs.

2.5.1 Probability Monads

Instead of directly working with probabilistic denotations of programs, the interaction of a program with probabilities/distribution objects can be modelled directly with *distribution monads* (equivalently, *probability*, or (to reach the continuous case) *Giry*, monads). This is indeed the representation that `math-comp/analysis` and `EasyCrpyt/certicrpyt` rely on. Further direct exposition is well described by Haskell's `monad-bayes` as well.

Briefly, at a high-level, probability distributions such as out $\mathcal{P}^{\mathcal{D}}$ can be seen to be well-modelled by monads by the following assignments:

$$\eta : \mathcal{D} \to \mathcal{P}^{\mathcal{D}}$$
$$\mu : \mathcal{P}^{\mathcal{P}^{\mathcal{D}}} \to \mathcal{P}^{\mathcal{D}}$$

Where η (unit), creates point masses[8] and μ (join), collapses distributions of distributions to distributions[9].

The choice of η for implementation of the probability monad is an object construction that usually directly corresponds to the data construction used to represent distributions. The choice of μ, on the other hand, embodies not just algebraically obvious reductions, but complex tradeoffs between the computational feasibility and numerical accuracy of inference on the epresented programs.

[8] alternatively interpreted as *dirac deltas*, or geometrically, as indexing into the vertex set of the simplex $\mathcal{P}^{\mathcal{D}}$

[9] the feasibility of which is also described as the "law of total probability"

Our artifact `../operationalize.ml` is an example of a domain in which $\mathcal{D}$ is sufficiently simple that both η and μ have an obvious efficient implementation. We defer to working implementations above for reasoing through this tradeoff in broader domains.

2.5.2 ITrees

Shifting from a direct representation of the operation of $\mathbb{AIM}$ programs as transforms on $\mathcal{P}^{\mathcal{D}}$ to a monadic representation reduces most proof goals from numerical mathematics to symbolic aplication of the monad laws. Indeed, to most users, the monad laws likely represent more accessible (intensional) probabilistic intuition than the laws of summation, convulution, and enumeration relief upon in more "direct" (extensional) representations of probability.

But the reduction of $\mathbb{AIM}$ semantic analysis to manipulation of monad rules and operators relies upon the specific representation of probability and probabilistic inference as a monad functor with accopanying implementations of η and μ. Whereas, in theory, equivalence of $\mathbb{AIM}$ programs should not depend on the particular choice of probability monad.

This motivates an elegant, modern presentation of the semantics of programs operating against a monad: *interaction trees*.

Instead of denotationalizing programs as specific instances of monadic computation, *itrees* assume only the existence of a *free monad*[10], and constructs programs not as particular inductive instances of the monad, but as coinductive instances of procedures that progress[11] through a mix of "internal" computation steps (i.e. computation that does not depend on the monad) and external "interaction" steps (that split the program into a prequel that terminates in a "request" to the monad, and a sequel continuation that depends on the "result" of that request).

With programs and environments defined in this itree style, programs, as ASTs of requests to the environment, and environments, as stateful servers of those requests, can be seperately analyzed, then compiled together via a semantics-preserving "handler" that matches requests and responses.

2.5.3 Itree Equivalence

ITrees are powerful not just as an elegant representation of the interface between programs and their environment, but as a framework to refine program/environment semantics via equivalence with respect to that interface.

Since ITrees represent interactions as explicit syntactic nodes (Visible events), we can establish semantic equivalence using weak bisimulation ("Equivalence Up To Tau" or `eutt`). This allows us to prove two programs are equivalent by structurally matching their external interactions, safely ignoring any finite number of internal, silent computation steps (Tau nodes).

[10] i.e. a monad with no properties except those common to all monads

[11] to an (optional) point of termination

In practice, equivalence up to tau can often be derived from the compounded application of simple rewrite rules. The rewrite rules, in turn, can be justified from the free semantics of itrees in combination with refining properties of the program and environment.

Proofs that depend only on equivalence up to tau are thus stable in a very general way: any changes to the program or environment that do not invalidate the properties relied upon to justify the set of itree equivalence rewrite rules do not invalidate the proof. Equivalently, we can say that itrees enable seperate reasoning about programs and their environment, or about program syntax and semantics.

2.5.4 Applications of ITree Equivalence to $\mathbb{AIM}$ Programs

The environment in which $\mathbb{AIM}$ programs operate is the universe/meta-universe pair $(\mathcal{U}, \Theta)$. Yet the inference rules above defining operational semantics for $\mathbb{AIM}$ program rely on $\mathcal{U}$ and Θ only through the functions $\mathcal{O}$ and $\mathcal{I}$. This allows $(\mathcal{O}, \mathcal{I})$ to form the signature of interaction between program and environment in an itree-based characterization of $\mathbb{AIM}$ program semantics.

As a powerful motivating example, consider the following theorem:

$$\mathbb{A}_Z \mathbb{A}_X \mathbb{M}_Y \cong_\tau \mathbb{A}_Z \mathbb{I}_X \mathbb{M}_Y \iff \text{no-backdoor}(X \to Y \mid Z)$$

which draws a logical equivalence between certain program equivalencies and corresponding properties of the underlying causal graph. Refinements of this theorem to particular programs (experimental procedures) and environments (causal graphs) are a classic "hello world" of causal analysis.

In this form though, the proof applies not just to particular instances of programs and environments, but to broad enough classes of program and environment that it functions as a *verified optimization* in a general compilation pipeline.

For many properties such as the rewrite presented above, the dependence on the causal graph is mediated through a predicate "`no-backdoor`" (or equivalently, through conditional independencies). Prior work[12] shows that mechanized characterization of this predicate is practical.

3 Binary Hypothesis Testing

Though much of this report is intended to form the basis for a broad and rich space of mathematical analyses whose complexity exceeds the progress made in their exploration, the case of *boolean/binary classification* lends itself to a straightforwards operationalize whose evaluation represents the first empirical evidence of the utility of $\mathbb{AIM}$ programs.

In particular, given two $\mathbb{AIM}$ programs, $\mathcal{R}_{\text{test}}, \mathcal{R}_{\text{train}}$, derive a procedure for estimating the results of $\mathcal{R}_{\text{test}}$ given access to the results of $\mathcal{R}_{\text{train}}$. Further,

[12] Anna's thesis/PLDI '26 paper

assume that

$$\mathcal{D}_{\text{test}} \equiv \mathbb{B} \equiv \{0, 1\} \equiv \{H_1, H_0\}$$

i.e., $\mathcal{R}_{\text{test}}$ is a "binary hypothesis test" - whose values can be characterized by a single bit, and whose outcomes are often referred to as H_1 (effect) and H_0 (null hypothesis).

3.1 Liklihood Ratio Tests

Now, even without a prior on Θ, we can define the *likelihood ratio*:

$$\forall d \in \mathcal{D}_{\text{train}} : \mathcal{L}(d) := \frac{\mathbb{P}[\mathbb{F}(\mathcal{R}_{\text{train}}) = d \mid \mathbb{F}(\mathcal{R}_{\text{test}}) = H_1]}{\mathbb{P}[\mathbb{F}(\mathcal{R}_{\text{train}}) = d \mid \mathbb{F}(\mathcal{R}_{\text{test}}) = H_0]}$$

some of the details of computing this test across the whole domain $\mathcal{D}_{\text{train}}$ (which is not constrained to be binary - and in fact benefits greatly (as will be seen below) from increased multiplicty) are elided for now.

Intuitively, we can effectively discriminate amongst the hypotheses $\mathcal{R}_{\text{test}} \sim \{H_1, H_0\}$ by running $\mathcal{R}_{\text{train}}$ to obtain a result d and computing its liklihood ratio $0 \leq \mathcal{L}(d) \leq \infty$. To utilize this derived variable, we note that larger values of $\mathcal{L}(d)$ correspond to increased confidence in H_1 and smaller values to increased confidence in H_0. This means[13] that no test that predicts H_1 for a value of $\mathcal{L}$ less than a value for which it predicts H_0 can be optimal (switching the two predictions would provide uniform improvement), so optimal tests always take the form of a *liklihood ratio test*:

$$d \mapsto \mathcal{L}(d) \geq \eta \implies H_1$$
$$d \mapsto \mathcal{L}(d) < \eta \implies H_0$$

Where $\eta \geq 0$ is called the *threshold* of the test.

3.2 Neyman-Pearson Frontiers

[14]

In the bayesian setting, there exist derivations of optimal choice of η for any well-behaved cost criterion (alt. loss function). In general, choice of η yields a tradeoff betwen the following two quantities:

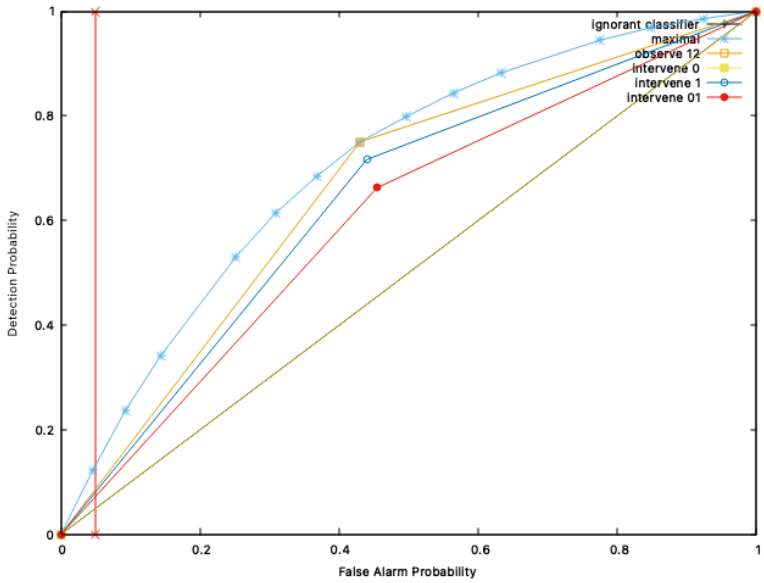
$$P_f = \mathbb{P}[\hat{H}_1 \mid H_0]$$
$$P_d = \mathbb{P}[\hat{H}_1 \mid H_1]$$

P_f, the "false alarm probability", is the chance of determing the effect is present when it is not. P_d, the "detection probability", is the chance of determining the effect is present when it is.

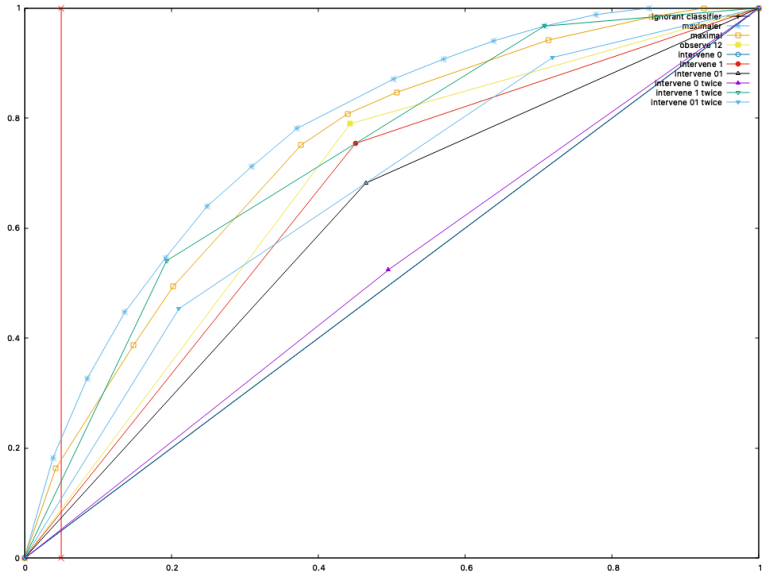
[13] a result classical called the "Neyman–Pearson lemma"

[14] Also called ROC (receiver operating characteristic) curves

Increasing η decreases the sensitivity of the test, which lowers both P_f and P_d. This yields the following graphical illustration of the tradeoff:



This shows the (P_f, P_d) characteristic for varying η for various experiments. Further details to be expounded upon soon.



This shows the frontiers for a wider range of programs, whose definitions in our ocaml version of this math is also provided below (for loose reference). In particular, this latter graph shows that not all estimators exhibit uniform prefernetiablity.

```
let () = [
    ("maximaler", [Measure 0; Measure 1; Measure 2;
                   Intervene (true, 0); Intervene (true, 1); Measure 2;
                   Intervene (false, 0); Intervene (true, 1); Measure 2;
                   Intervene (true, 0); Intervene (false, 1); Measure 2;
                   Intervene (false, 0); Intervene (false, 1); Measure 2]
    );

    ("maximal", [Measure 0; Measure 1; Measure 2;
                 Intervene (true, 0); Intervene (true, 1); Measure 2;
                 Intervene (false, 0); Intervene (true, 1); Measure 2]);

    ("observe 12", [Measure 1; Measure 2]);

    ("intervene 0", [Intervene (true, 0); Measure 2]);

    ("intervene 1", [Intervene (true, 1); Measure 2]);

    ("intervene 01", [Intervene (true, 1); Intervene (true, 0); Measure 2]);

    ("intervene 0 twice", [Intervene (true, 0); Measure 2; Intervene (false, 0); Measure 2]);

    ("intervene 1 twice", [Intervene (true, 1); Measure 2; Intervene (false, 1); Measure 2]);

    ("intervene 01 twice", [Intervene (true, 1); Intervene (true, 0); Measure 2;
                            Intervene (false, 1); Intervene (false, 0); Measure 2])


  ] |> List.map (fun (title, prog) -> (title, pf_pd reccomend_one_for_two_signal prog))
        |> plot_scatter

(* to run:
   dune exec ./operationalize.exe
 *)
```